

PoorChat

Plan técnico, división de trabajo y tareas secuenciales.

Proyecto de aplicación cliente-servidor para mensajería, transferencia de archivos, llamadas y videollamadas usando sockets, TCP, UDP, POO, patrones de diseño y diagramas UML.

Fecha de referencia: 16 de junio de 2026.

1. Estado actual del proyecto

El repositorio ya tiene una estructura base por carpetas. La aplicación está orientada a Electron: src/main.js abre la ventana principal y carga src/ui/index.html. La mayoría de módulos técnicos todavía están vacíos, por lo que es buen momento para definir responsabilidades antes de implementar.

Rutas actuales importantes:

- src/main.js: proceso principal de Electron. Crea la ventana y carga la interfaz.
- src/ui/: interfaz, renderer, preload y estilos. Actualmente solo hay HTML inicial.
- src/server/: servidor TCP, manejo de clientes y ruteo de mensajes. Todavía vacío.
- src/network/: abstracciones de conexión TCP, UDP y configuración de puertos. Todavía vacío.
- src/handlers/: lógica de mensajes de texto y archivos. Todavía vacío.
- src/audio/ y src/video/: envío y recepción de audio/video por UDP. Todavía vacío.
- src/util/: funciones auxiliares de tiempo, archivos y validación. Todavía vacío.

La estructura está bien como punto de partida, pero todavía falta separar responsabilidades centrales: cliente, protocolo, modelos y autenticación.

2. Idea de arquitectura

La aplicación debe separar claramente el tráfico confiable del tráfico en tiempo real.

- TCP: login, mensajes, archivos, historial, control de sesión y control de llamadas.
- UDP: audio y video en tiempo real.
- POO: usuarios, mensajes, sesiones, transferencias de archivos y llamadas.
- Patrones de diseño: Observer/EventEmitter, Factory, Strategy, Singleton/Config y Command.
- UML: clases, secuencias de login, mensaje, archivo y llamada.

Comunicación principal por TCP:

```

Cliente Electron 1
Cliente Electron 2
Cliente Electron 3
|
| TCP: login, mensajes, archivos, control de llamadas
v
Servidor central Node.js
^
|
| TCP
Cliente Electron N
```

Comunicación multimedia por UDP:

Cliente A <----- UDP audio/video -----> Cliente B

El inicio de una llamada no debe ocurrir directamente por UDP. Primero se negocia por TCP: un cliente solicita llamar, el servidor avisa al destinatario, el destinatario acepta o rechaza y luego ambos clientes abren el canal UDP para enviar paquetes multimedia.

Cliente A -TCP-> Servidor: CALL_REQUEST a Cliente B
Servidor -TCP-> Cliente B: llamada entrante
Cliente B -TCP-> Servidor: CALL_ACCEPT
Servidor -TCP-> Ambos: datos de sesión
Cliente A <-UDP-> Cliente B: audio/video

3. División recomendada para 3 integrantes

Integrante 1: Servidor, TCP y login

Este integrante debe construir el núcleo del sistema. Su objetivo es que varios clientes puedan conectarse, autenticarse y comunicarse mediante el servidor.

Archivos principales:

- src/server/server.js
- src/server/clientHandler.js
- src/network/tcpConnection.js

Carpetas sugeridas:

- src/auth/
- src/models/
- src/protocol/

Clases sugeridas:

- Server
- ClientHandler
- SessionManager
- User
- AuthService
- MessageRouter

Responsabilidades:

- Crear servidor TCP con net.createServer.
- Manejar varios clientes conectados.
- Implementar login básico.
- Mantener una tabla de usuarios conectados.
- Enrutar mensajes de un cliente a otro.
- Validar que solo usuarios autenticados puedan enviar mensajes.

Patrones recomendados:

- Singleton para configuración de puertos.
- Observer/EventEmitter para eventos de conexión, mensaje y desconexión.
- Factory para crear mensajes según su tipo.
- Command para mapear tipos de mensaje a acciones del servidor.

Integrante 2: Cliente, UI, mensajes y archivos

Este integrante debe encargarse de la experiencia de usuario y de conectar la interfaz con la capa TCP. Debe trabajar con cuidado la separación entre renderer, preload y lógica Node.

Archivos principales:

- src/ui/index.html
- src/ui/renderer.js
- src/ui/preload.js
- src/handlers/textHandler.js
- src/handlers/fileHandler.js
- src/util/fileUtil.js

Carpeta sugerida:

- src/client/

Clases sugeridas:

- ChatClient
- TextHandler
- FileHandler
- FileTransfer
- Message

Responsabilidades:

- Pantalla de login.
- Pantalla de chat.
- Conexión TCP desde el cliente.
- Envío y recepción de mensajes.
- Selección y envío de archivos.
- Guardado de archivos recibidos.
- Integración segura con Electron usando preload.js e IPC.

Regla importante: el renderer de Electron no debería usar Node directamente. Lo correcto es:

```
renderer.js -> preload.js -> main.js / cliente TCP
```

Integrante 3: UDP, audio, video y llamadas

Este integrante debe encargarse de la parte más experimental del proyecto. Conviene empezar con pruebas UDP simples y luego avanzar hacia audio y video reales.

Archivos principales:

- src/network/udpConnection.js
- src/audio/audioSender.js
- src/audio/audioReceiver.js
- src/video/videoSender.js
- src/video/videoReceiver.js

Clases sugeridas:

- UdpConnection
- CallSession
- AudioSender
- AudioReceiver
- VideoSender
- VideoReceiver

- MediaPacket

Responsabilidades:

- Crear socket UDP con dgram.
- Implementar handshake de llamada.
- Enviar paquetes de audio.
- Recibir y reproducir audio.
- Enviar video en baja resolución/FPS.
- Controlar pérdida o desorden de paquetes usando sequence y timestamp.

Para video por UDP, lo más seguro es empezar simple: capturar frames desde la cámara, comprimirlos como JPEG/WebP, dividirlos en paquetes pequeños y reconstruirlos del otro lado. Primero prueben UDP con texto o audio simulado, luego audio real, luego video.

4. Estructura recomendada

La estructura actual puede evolucionar sin romper lo existente. Lo más importante es agregar carpetas para protocolo, modelos, cliente y autenticación.

```
src/  
main.js  
  
client/  
chatClient.js  
clientState.js  
  
server/  
server.js  
clientHandler.js  
sessionManager.js  
messageRouter.js  
  
network/  
tcpConnection.js  
udpConnection.js  
puertos.js  
  
protocol/  
messageTypes.js  
tcpProtocol.js  
udpProtocol.js  
  
models/  
User.js  
Message.js  
FileTransfer.js  
CallSession.js  
  
handlers/  
textHandler.js  
fileHandler.js  
callHandler.js  
  
auth/  
authService.js  
userRepository.js  
  
audio/  
audioSender.js  
audioReceiver.js  
  
video/
```

```
videoSender.js  
videoReceiver.js
```

```
util/  
timeUtil.js  
fileUtil.js
```

```
ui/  
index.html  
styles.css  
renderer.js  
preload.js
```

5. Protocolo base

Antes de programar muchas pantallas, conviene definir el formato de mensajes. Esto permite que servidor, cliente y handlers avancen en paralelo usando el mismo contrato.

Mensaje TCP sugerido:

```
{  
  "type": "LOGIN",  
  "payload": {  
    "username": "carlos",  
    "password": "1234"  
  }  
}
```

Tipos TCP mínimos:

- LOGIN
- LOGIN_OK
- LOGIN_ERROR
- TEXT_MESSAGE
- FILE_START
- FILE_CHUNK
- FILE_END
- CALL_REQUEST
- CALL_ACCEPT
- CALL_REJECT
- CALL_END
- ERROR

Transferencia de archivos por TCP:

```
FILE_START: nombre, tamaño, tipo, emisor, receptor, transferId  
FILE_CHUNK: transferId, chunkIndex, bytes  
FILE_END: transferId
```

Paquete UDP sugerido:

```
{  
  "callId": "abc123",  
  "mediaType": "audio",  
  "sequence": 150,  
  "timestamp": 17100000000000,  
  "chunkIndex": 0,  
  "totalChunks": 1,  
  "payload": "bytes..."  
}
```

Para UDP se recomienda mantener paquetes pequeños, idealmente por debajo de 1200 bytes cuando sea posible. Para video, un frame comprimido puede dividirse en varios paquetes y reconstruirse usando sequence, chunkIndex y totalChunks.

6. Patrones de diseño aplicables

- Observer/EventEmitter: emitir eventos cuando llega un mensaje, archivo, solicitud de llamada o desconexión.
- Factory Method: crear objetos de mensaje según el campo type.
- Strategy: elegir transporte, TCP para mensajes/archivos y UDP para audio/video.
- Singleton/Config: centralizar puertos, host, rutas y constantes de la aplicación.
- Command: mapear cada tipo de mensaje TCP a una acción específica del servidor.

7. Diagramas UML necesarios

1. Diagrama de clases: Server, ClientHandler, ChatClient, User, Message, FileTransfer, CallSession, TcpConnection, UdpConnection.
2. Diagrama de secuencia de login.
3. Diagrama de secuencia de mensaje de texto.
4. Diagrama de secuencia de transferencia de archivo.
5. Diagrama de secuencia de llamada o videollamada: señalización por TCP y media por UDP.
6. Diagrama de componentes: UI, cliente, servidor, protocolo, handlers, audio/video.

8. Tareas secuenciales para completar el proyecto

Estas tareas están ordenadas por dependencia. No conviene saltar directamente a video o UI final sin tener primero el protocolo, el servidor y la mensajería básica.

Fase 0: Organización inicial

1. Crear ramas de trabajo para cada integrante: server-tcp-login, client-ui-files, udp-calls-media.
2. Agregar carpetas faltantes: src/client, src/protocol, src/models, src/auth.
3. Definir convenciones: nombres de clases, formato de commits, puertos y estilo de mensajes JSON.
4. Crear un README con instrucciones para ejecutar servidor y cliente.

Fase 1: Protocolo común

1. Crear messageTypes.js con todos los tipos TCP y UDP.
2. Crear tcpProtocol.js con funciones para serializar y parsear mensajes JSON.
3. Definir cómo separar mensajes TCP. Recomendación: JSON por línea usando \n como delimitador.
4. Crear validadores mínimos para evitar mensajes sin type o sin payload.
5. Documentar ejemplos de cada mensaje en el README.

Fase 2: Servidor TCP mínimo

1. Implementar server.js con net.createServer.
2. Implementar clientHandler.js para manejar conexión, datos recibidos y desconexión.
3. Crear una tabla en memoria de clientes conectados.
4. Agregar logs claros para conexión, mensaje recibido y desconexión.
5. Probar con clientes simples por consola antes de usar Electron.

Fase 3: Login y sesiones

1. Crear AuthService y UserRepository.
2. Empezar con usuarios en memoria o archivo JSON local.
3. Implementar mensajes LOGIN, LOGIN_OK y LOGIN_ERROR.
4. Guardar en SessionManager qué socket pertenece a qué usuario.
5. Bloquear mensajes de usuarios no autenticados.

Fase 4: Cliente TCP y UI de login

1. Crear ChatClient para conectar al servidor por TCP.
2. Exponer funciones seguras desde preload.js hacia renderer.js.
3. Diseñar pantalla de login simple.
4. Mostrar estado de conexión: desconectado, conectando, conectado, error.
5. Probar login desde la interfaz Electron.

Fase 5: Mensajes de texto

1. Implementar TEXT_MESSAGE en el cliente.
2. Implementar ruteo de mensajes en el servidor usando MessageRouter.
3. Agregar lista de usuarios conectados.
4. Mostrar mensajes enviados y recibidos en la UI.
5. Probar con dos o más clientes conectados al mismo servidor.

Fase 6: Transferencia de archivos por TCP

1. Crear FileTransfer con id, nombre, tamaño, tipo, emisor y receptor.
2. Implementar FILE_START, FILE_CHUNK y FILE_END.
3. Enviar archivos por chunks, no como un JSON gigante.
4. Guardar archivos recibidos en una carpeta definida, por ejemplo downloads/.
5. Probar documentos, imágenes, audio y video pequeños.
6. Agregar progreso de envío y recepción en la UI.

Fase 7: Señalización de llamadas por TCP

1. Crear CallSession con callId, emisor, receptor, estado y puertos UDP.
2. Implementar CALL_REQUEST, CALL_ACCEPT, CALL_REJECT y CALL_END.
3. Mostrar llamada entrante en la UI.
4. Permitir aceptar, rechazar y colgar.
5. Confirmar que ambos clientes reciben los datos necesarios para iniciar UDP.

Fase 8: UDP básico

1. Implementar udpConnection.js usando dgram.
2. Enviar paquetes UDP de prueba entre dos clientes.
3. Agregar sequence y timestamp a cada paquete.
4. Medir pérdida o desorden de paquetes con logs.
5. Probar con IPs de Tailscale.

Fase 9: Audio por UDP

1. Capturar audio desde el cliente.
2. Dividir audio en paquetes pequeños.
3. Enviar audio por UDP con AudioSender.
4. Recibir y reproducir audio con AudioReceiver.
5. Agregar buffer pequeño para suavizar cortes.

6. Probar llamada de audio entre dos clientes.

Fase 10: Video por UDP

1. Capturar frames desde cámara o pantalla.
2. Comprimir frames con calidad baja o media.
3. Dividir frames grandes en chunks UDP.
4. Reconstruir frames en el receptor.
5. Mostrar video remoto en la UI.
6. Reducir resolución o FPS si la red se satura.

Fase 11: Integración con Tailscale

1. Instalar Tailscale en los equipos de prueba.
2. Identificar la IP Tailscale del equipo servidor.
3. Configurar el servidor para escuchar en 0.0.0.0.
4. Configurar clientes para conectarse a la IP Tailscale del servidor.
5. Probar TCP primero: login, mensajes y archivos.
6. Probar UDP después: paquetes de prueba, audio y video.

Fase 12: UML y documentación final

1. Actualizar diagrama de clases con las clases reales implementadas.
2. Crear diagramas de secuencia de login, mensaje, archivo y llamada.
3. Crear diagrama de componentes.
4. Documentar cómo ejecutar servidor y cliente.
5. Documentar limitaciones conocidas: pérdida UDP, tamaño de archivos, seguridad del login.

Fase 13: Pruebas y cierre

1. Probar un cliente en la misma máquina que el servidor.
2. Probar dos clientes en la misma red.
3. Probar clientes por Tailscale.
4. Probar desconexiones inesperadas.
5. Probar login incorrecto, usuario duplicado, archivo grande y llamada rechazada.
6. Preparar demo final con un flujo completo: login, mensaje, archivo, llamada y videollamada.

9. Orden de prioridad

Si el tiempo es limitado, el orden mínimo viable debe ser:

1. Protocolo TCP.
2. Servidor multi-cliente.
3. Login.
4. Mensajes de texto.
5. Archivos por TCP.
6. Señalización de llamadas por TCP.
7. UDP simple.
8. Audio.
9. Video.
10. UML final y documentación.

La recomendación principal es no empezar por video. Primero deben tener una base sólida de servidor, protocolo, login y mensajes. Con eso funcionando, archivos y llamadas se vuelven extensiones naturales en lugar de problemas mezclados.